

EE5176 Assignment 1: Basics of Imaging Demosaicing, White Balancing, and Tone Mapping

Amritam Halder EE23B004

September 27, 2026

1 Motion Deblurring with Conventional Camera

The clean input image has dimensions

$$M \times N \times 3 = 450 \times 800 \times 3.$$

The exposure consists of 52 instants, $t = 0, \dots, 51$. At each instant the image is translated horizontally by t pixels. The captured image is obtained by averaging the translated images. Gaussian noise from `gaussNoise.mat` is then added, with the noise converted from the given 255-scale representation to the normalized image range used in the notebook.

1.1 Part 1(a): Blurred and Noisy Image

For each t , the translated image is padded on the left and right and the 52 translated versions are averaged:

$$B = \frac{1}{52} \sum_{t=0}^{51} T_t(I).$$

The noisy observation is

$$Y = B + n,$$

where n is the Gaussian noise.



Figure 1: Conventional-camera result: blurred image obtained by averaging 52 horizontal translations and the corresponding image after adding Gaussian noise.

The output shows the expected horizontal motion blur. Adding the Gaussian noise produces visible pixel-level fluctuations over the blurred image, while preserving the overall shape and colors of the scene.

1.2 Part 1(b): Blur Matrix

Because the blur is a uniform horizontal average, the blur operator is a Toeplitz matrix. For one row, the observation model can be written as

$$\mathbf{y} = A\mathbf{x},$$

where A has dimensions

$$(N + 51) \times N = 851 \times 800.$$

Its first column contains 52 entries equal to $1/52$, followed by zeros, and the first row begins with $1/52$ followed by zeros.

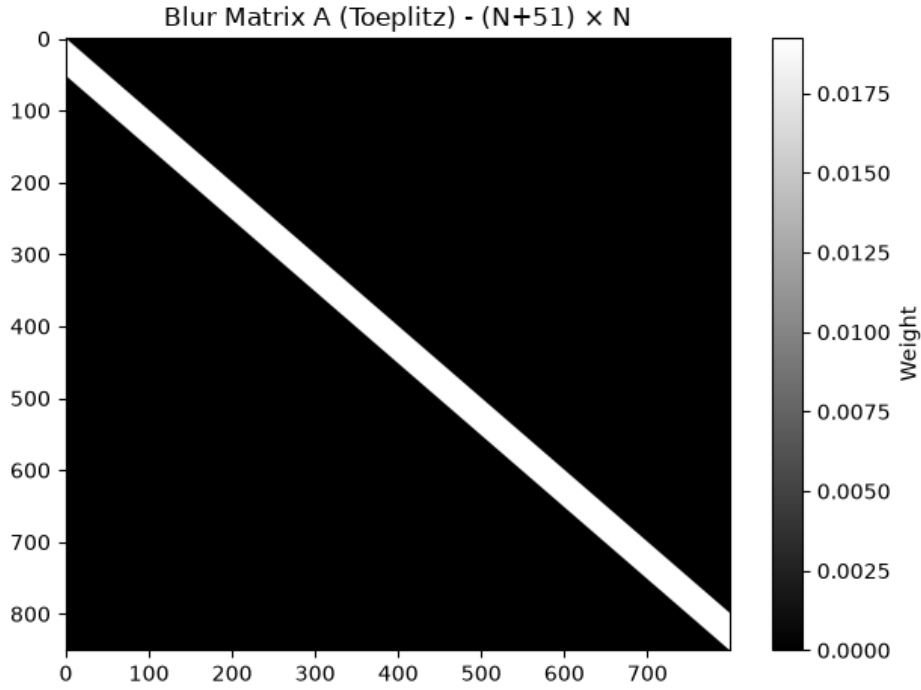


Figure 2: Toeplitz blur matrix A for the 52-frame conventional shutter.

The diagonal band in the visualization corresponds to the 52 translated copies contributing to each output position.

1.3 Part 1(c): Least-Squares Deblurring

Since the blur is horizontal, the inverse problem is solved independently for every row and every colour channel. The least-squares estimate is

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmin}} \|\mathbf{A}\mathbf{x} - \mathbf{y}\|_2^2.$$

Using the Moore-Penrose pseudoinverse,

$$\hat{\mathbf{x}} = A^\dagger \mathbf{y}.$$

The RMSE used in the notebook is

$$\text{RMSE} = \sqrt{\frac{1}{3MN} \sum_{c=1}^3 \sum_{i=1}^M \sum_{j=1}^N \left(I_{ijc} - \hat{I}_{ijc} \right)^2}.$$

The obtained RMSE is

$$\boxed{\text{RMSE} = 0.333941}.$$

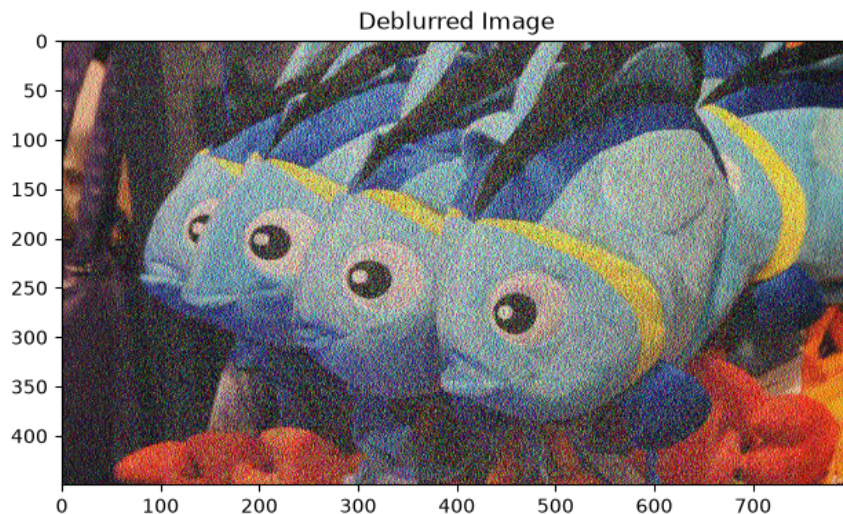


Figure 3: Deblurred result obtained using the pseudoinverse of the conventional blur matrix.

1.4 Part 1(d): Observation

The major structures of the fish are recovered and the horizontal blur is significantly reduced. However, the reconstruction contains strong grainy and coloured noise, particularly in smooth regions and near object boundaries.

The reason is that the moving-average blur suppresses high-frequency components. The pseudoinverse must amplify these weak components in order to undo the blur, but the same amplification also acts on the Gaussian noise. Thus,

$$A^\dagger(Ax + n) \approx x + A^\dagger n,$$

and the noise term can become large for components associated with small singular values of A . Therefore, the deblurred result becomes sharper but significantly noisier.

2 Motion Deblurring with Flutter Shutter

Two 52-second shutter codes were used:

Code 1 = 1010000111000001010000110011110111010111001001100111

and

Code 2 = 10.

Each code contains 26 active exposure instants. A translated image is included in the exposure average only when the corresponding bit is one.

2.1 Part 2(a): Flutter-Shutter Blur

For a binary shutter code s_t , the blurred image is

$$B_s = \frac{1}{\sum_t s_t} \sum_{t=0}^{51} s_t T_t(I),$$

followed by the same Gaussian-noise addition used in Question 1.

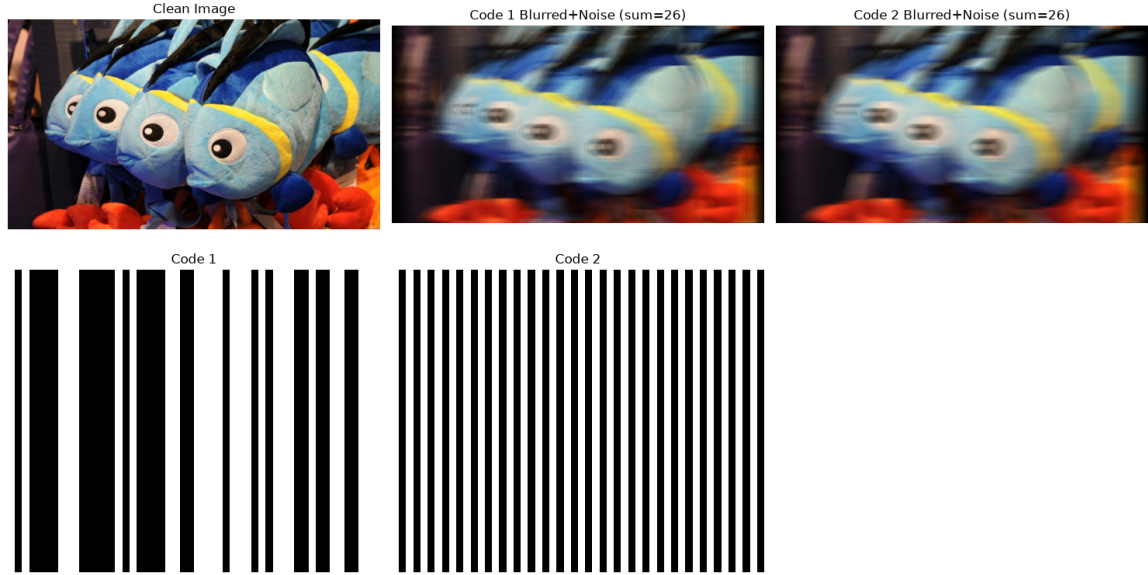


Figure 4: Clean image, noisy flutter-shutter outputs for Codes 1 and 2, and the corresponding shutter-code patterns.

The two shutter patterns produce visibly different blur patterns because the exposure times are distributed differently over the motion trajectory.

2.2 Part 2(b): Blur Matrices

The corresponding blur matrices are again Toeplitz, but their non-zero entries follow the active positions of the shutter codes.

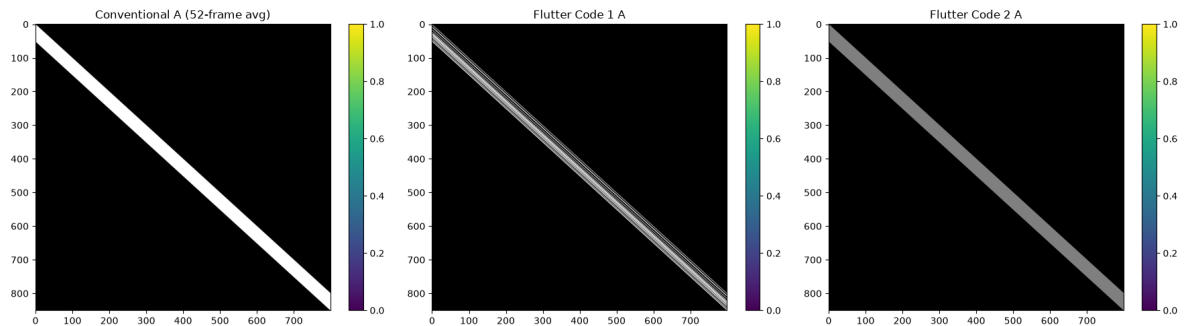


Figure 5: Toeplitz blur matrices for the conventional shutter, Flutter Code 1, and Flutter Code 2.

The matrices directly visualize the temporal shutter pattern: the conventional shutter has a

continuous band, whereas the flutter-shutter matrices have non-zero entries only at the active exposure instants.

2.3 Part 2(c): DFT Comparison

The first column of each blur matrix represents the corresponding one-dimensional shutter kernel. Its discrete Fourier transform characterizes the attenuation of different spatial frequencies.

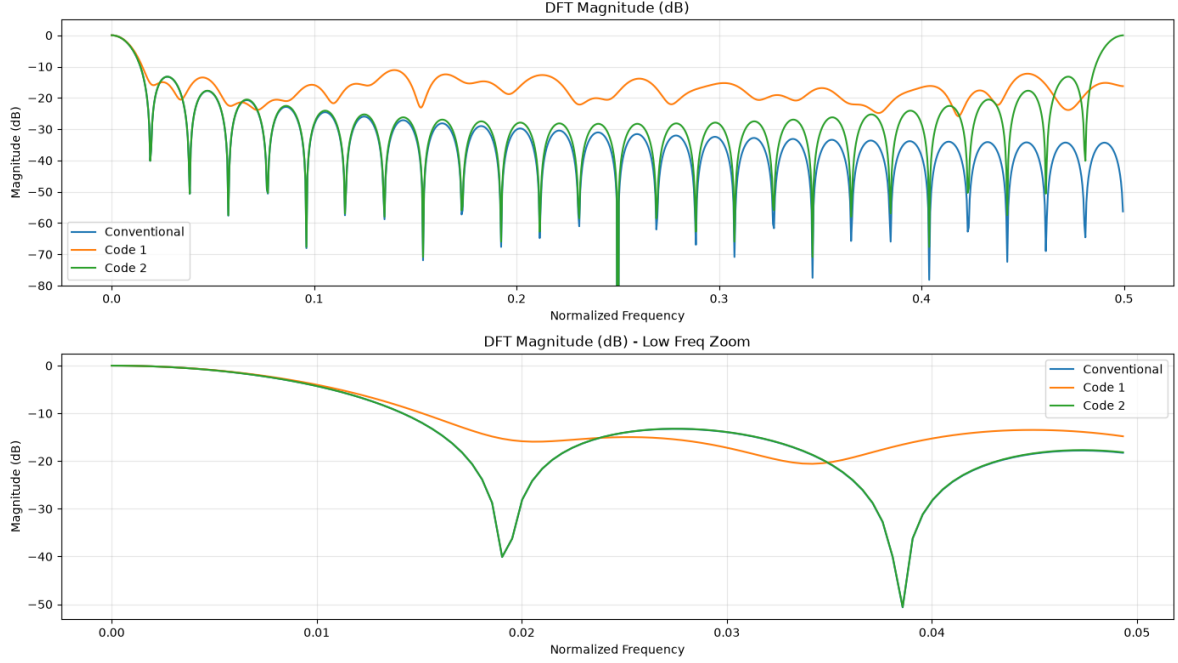


Figure 6: DFT magnitude in decibels for the conventional shutter and the two flutter-shutter codes.

The conventional 52-frame average has strong frequency nulls due to its box-filter nature. Flutter shutter changes the spectral response and can prevent severe attenuation at a larger number of frequencies. In the notebook, Code 1 gives the best conditioning among the three constructed blur matrices.

2.4 Part 2(d): Least-Squares Deblurring

The least-squares estimate for each row and colour channel is again obtained by

$$\hat{\mathbf{x}} = A^\dagger \mathbf{y}.$$

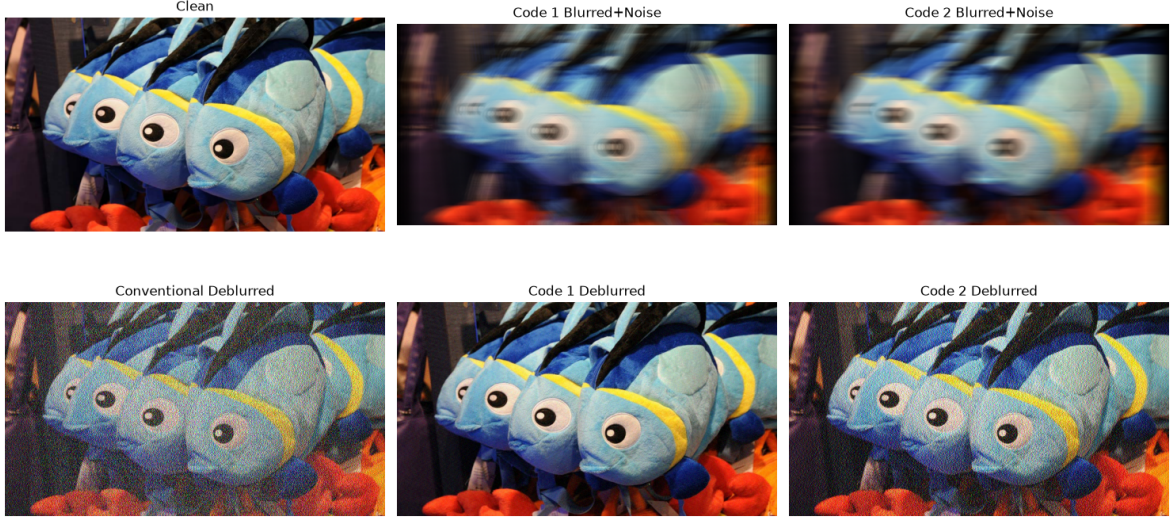


Figure 7: Comparison of the clean image, noisy blurred images, and least-squares deblurred outputs for the conventional shutter and the two flutter-shutter codes.

Code 1 produces a visually sharper reconstruction with substantially less noise than the conventional shutter. Code 2 improves over the conventional shutter but still exhibits noticeable artifacts.

2.5 Part 2(e): RMSE Comparison

Table 1: RMSE of the noisy deblurred outputs.

Method	RMSE
Conventional shutter	0.333941
Flutter Code 1	0.033469
Flutter Code 2	0.203550

Code 1 has the lowest RMSE and is visually closest to the clean image. Code 2 provides an intermediate result. The difference is explained by the different frequency responses and conditioning of the blur matrices. When the pseudoinverse is applied, frequencies for which the blur transfer function is small are strongly amplified. Flutter Code 1 preserves the relevant frequencies better and therefore amplifies the measurement noise much less than the conventional shutter.

2.6 Part 2(f): No-Noise Case

The same deblurring experiments were repeated after removing the Gaussian noise.

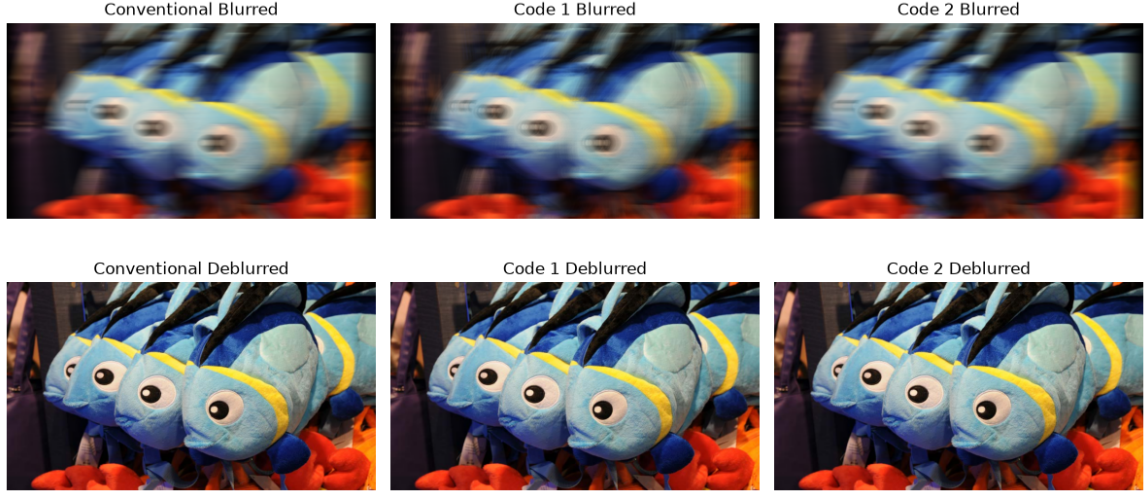


Figure 8: Deblurred outputs when the additive Gaussian noise is absent.

The RMSE values are

Table 2: RMSE in the absence of noise.

Method	RMSE
Conventional shutter	5.485×10^{-14}
Flutter Code 1	3.266×10^{-15}
Flutter Code 2	3.083×10^{-14}

Thus, the large differences observed in Part 2(e) are mainly caused by the interaction between the noise and the ill-conditioned inverse problem. In the noise-free case, the blur itself can be inverted very accurately.

3 Deblurring with Motion-Invariant Photography

In this section, the background and a foreground red car are considered separately. For the motion-invariant case, the camera translation values are supplied in `CameraT.mat`.

3.1 Part 3(a): Moving Car with Static Camera

The car is translated by one pixel per second to the right relative to a static camera. At each time instant, the translated car is merged with the stationary background, and the 52 frames are averaged.

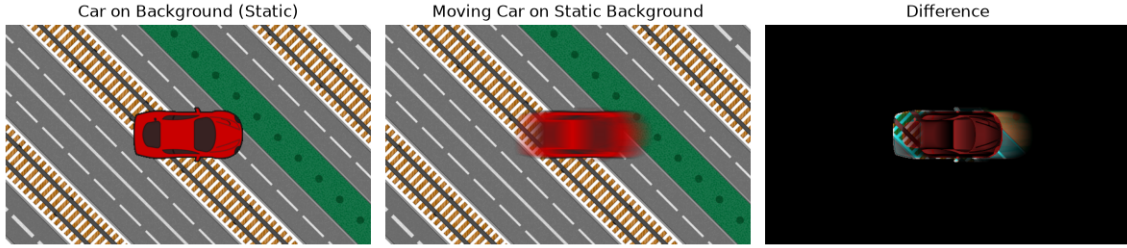


Figure 9: Static-camera case

The blurred car is spread mainly along the direction of motion, while the background remains fixed.

3.2 Part 3(b): Moving Camera and Moving Car

The translation values of the static background due to camera motion are loaded from the file `CameraT.mat`. The notebook then uses

$$\text{relative translation} = \text{camera translation} - \text{object translation}$$

for the foreground car, as prescribed in the assignment.

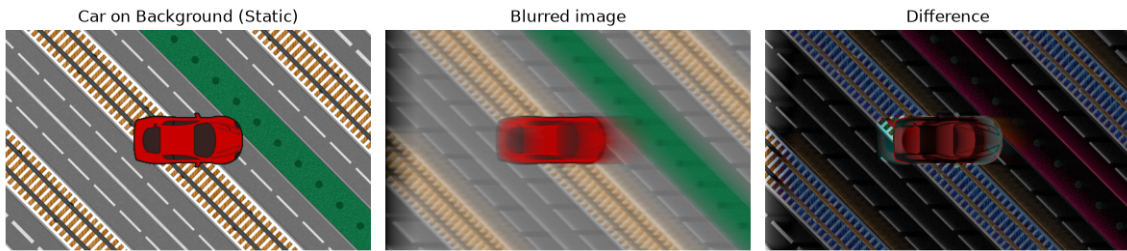


Figure 10: Camera Translation case

The supplied translation values vary symmetrically, reaching approximately zero near the centre and larger magnitudes toward the beginning and end of the sequence. Thus, the background and the car undergo different image-plane motions over the exposure.

3.3 Part 3(c): Comparison of (a) and (b)

The absolute difference between the two blurred scenarios is shown below.

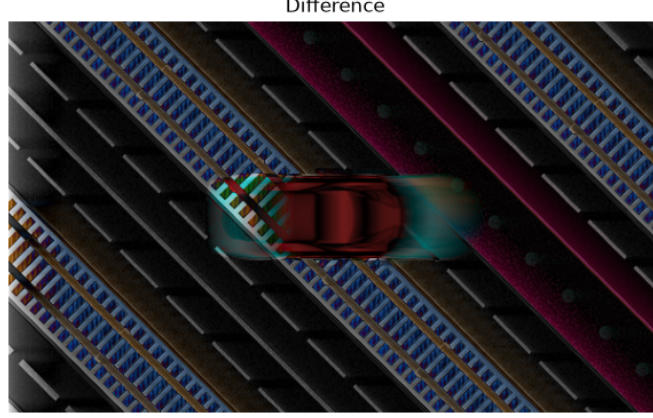


Figure 11: Difference between the static-camera and moving-camera blurred scenarios.

The RMSE between the two blurred scenarios is

$$\boxed{0.190049}.$$

In Part 3(a), the background remains sharp while the car alone moves. In Part 3(b), the camera motion causes the background itself to move in the image, while the car has a different relative motion with respect to the moving camera. Consequently, the overall blur is more complex and affects both the foreground and background.

3.4 Part 3(d): PSF of the Motion-Invariant Blur

The assignment specifies the PSF weight model

$$w(x) = \frac{1}{x^2},$$

with

$$x = \{0.1, 1, 2, \dots, 51\}.$$

The normalized PSF is therefore

$$h(x) = \frac{1/x^2}{\sum_k 1/x_k^2}.$$

Also checked with actual camera movement by using the `CameraT.mat` by sorting it out for the actual translation.

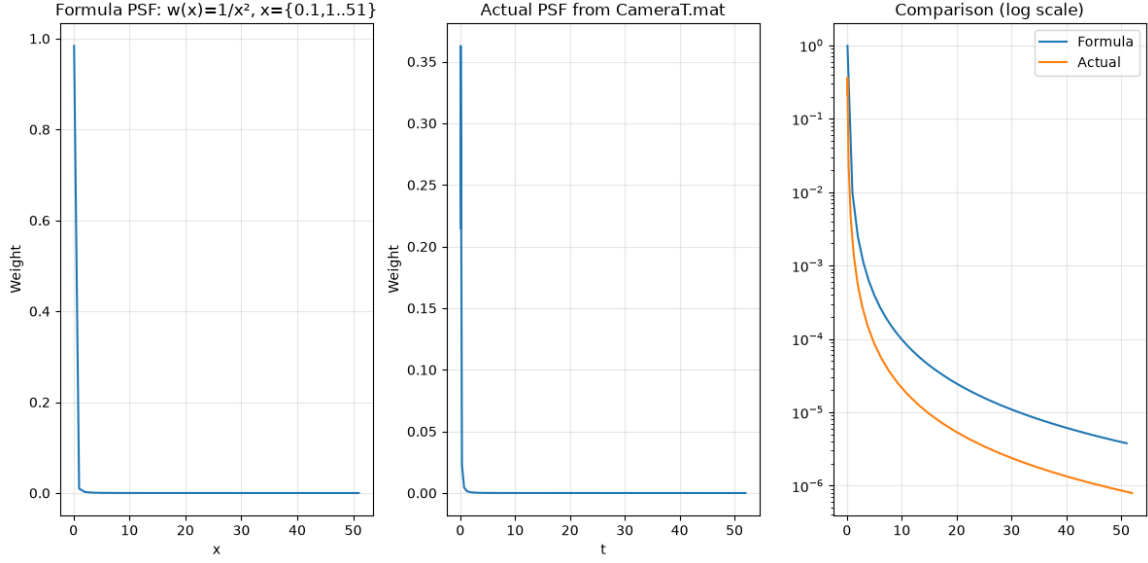
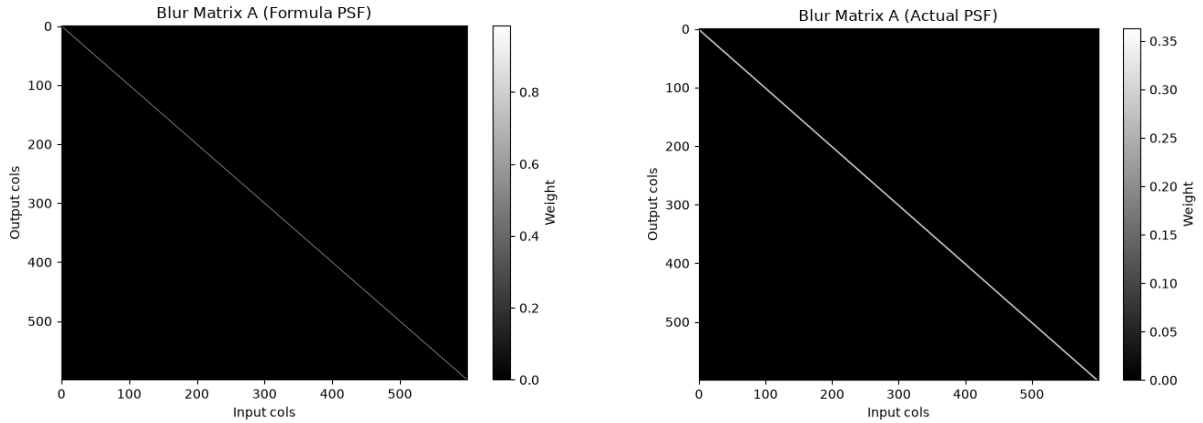


Figure 12: PSF constructed with both Given and actual translation.

The $x = 0.1$ value is explicitly specified so that the $1/x^2$ expression remains finite at the zero-translation point. Therefore, this PSF is a model-based PSF prescribed by the assignment rather than a direct estimation from the image.

3.5 Part 3(e): Blur Matrix from the PSF

The normalized PSF from Part 3(d) is embedded in a Toeplitz blur matrix. The resulting matrix represents convolution by the PSF along each image row.



(a) Blur matrix for Formula PSF.

(b) Actual trajectory-based PSF matrix.

Figure 13: Toeplitz blur matrices used in the Part 3 experiments. The formula-based matrix is the one required by the assignment.

3.6 Part 3(f): Least-Squares Deblurring

The blurred image from Part 3(b) is deblurred using the pseudoinverse of the blur matrix:

$$\hat{\mathbf{x}} = \mathbf{A}^\dagger \mathbf{y}.$$

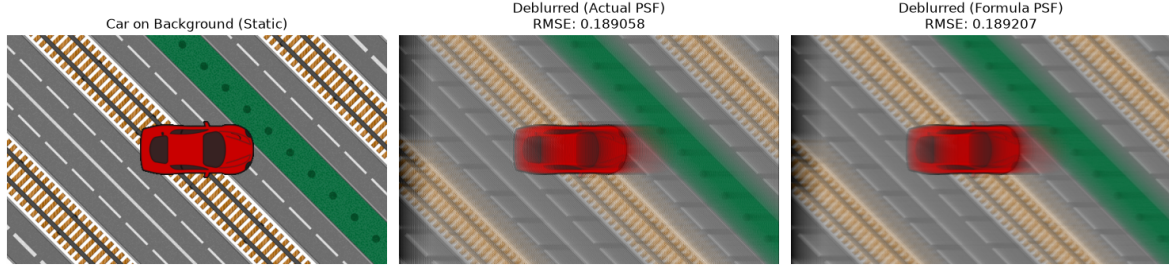


Figure 14: Comparison for deblurring.

The RMSE values are

$$\text{RMSE}_{\text{actual PSF}} = 0.189058,$$

and

$$\text{RMSE}_{\text{formula PSF}} = 0.189207.$$

The two results are numerically very close in this implementation. More importantly, neither reconstruction removes the full degradation introduced in Part 3(b). The reason is that the image in Part 3(b) is generated by two different motions: the background follows the camera trajectory, whereas the foreground car follows a different relative trajectory. A single spatially invariant convolution PSF is therefore only an approximation to the complete image formation process. The residual mismatch between the prescribed PSF model and the actual frame-generation process limits the improvement obtained from least-squares inversion.

4 References

The implementation was based on the assignment requirements and utilized only the permitted libraries:

- NumPy for numerical operations
- SciPy for mat file loading and toeplitz matrix
- Matplotlib for visualization
- Stack Exchange, Geeks for Geeks and Github for different ideas on the implementation

5 Self Declaration

I, Amritam Halder, swear on my honour that I have prepared and written the answers for this assignment and associated code by myself and have not copied from the internet, any LLM's output, or other students.